

Algorithmes de tri

Le tri est un problème classique (et utile) de l'algorithmique utilisé sur un ensemble d'éléments possédant une relation d'ordre total (comme l'ordre des nombres et l'ordre lexicographique pour les caractères) afin de l'ordonner dans l'ordre croissant (ou décroissant).

1 Critères d'évaluation des algorithmes de tri

Lorsqu'on choisit un algorithme de tri, on prend en compte sa :

- Complexité temporelle (vitesse du tri) : Mesure combien de temps l'algorithme met pour trier les éléments.
- Complexité spatiale (mémoire utilisée) : Évalue si les éléments sont triés sur place : ils sont réorganisés à l'intérieur de la structure utilisée sans mémoire supplémentaire.
- Stabilité : Un tri est stable s'il conserve l'ordre des éléments égaux.

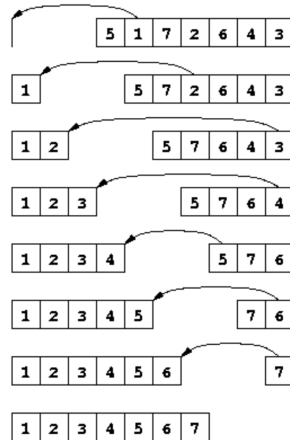
En plus, Un tri est dit en ligne si on peut commencer le tri avant même d'avoir reçu l'intégralité des données. Cela le rend particulièrement utile pour le traitement en flux, où les données sont reçues et doivent être triées progressivement.

2 Tri par comparaison

2.1 Tri par sélection

Le tri par sélection est un tri en place et non stable. Ce tri fonctionne en sélectionnant l'élément le plus petit de la partie non triée et en le permutant avec le premier élément de cette partie. Il continue ensuite avec le reste des éléments (en ignorant la partie déjà triée) jusqu'à ce que tout soit trié.

2.1.1 Illustration



2.1.2 Exemple de code

```
[1]: def tri_par_sélection(liste):  
    for i in range(len(liste)-1):  
        indice_minimum = liste.index(min(liste[i:]))  
        liste[i], liste[indice_minimum] = liste[indice_minimum], liste[i]  
        print(liste)  
    return liste  
  
liste = [5,1,7,2,6,4,3]  
tri_par_sélection(liste)
```

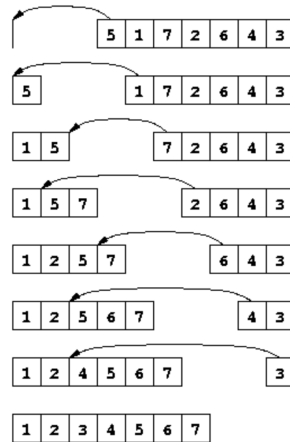
```
[1, 5, 7, 2, 6, 4, 3]  
[1, 2, 7, 5, 6, 4, 3]  
[1, 2, 3, 5, 6, 4, 7]  
[1, 2, 3, 4, 6, 5, 7]  
[1, 2, 3, 4, 5, 6, 7]  
[1, 2, 3, 4, 5, 6, 7]
```

```
[1]: [1, 2, 3, 4, 5, 6, 7]
```

2.2 Tri par insertion

Le tri par insertion est effectué en insérant chaque élément reçu à sa bonne position dans la partie triée. Il fonctionne comme lorsque l'on tient des cartes à jouer : on prend sur la table les cartes, une par une, pour l'insérer à la bonne position dans la main gauche en examinant les cartes, déjà triées, de la droite vers la gauche.

2.2.1 Illustration



2.2.2 Exemple de code

```
[2]: def tri_par_insertion(liste):
    for i in range(1, len(liste)):
        to_insert = liste[i]
        where = i-1
        while to_insert < liste[where] and where >= 0:
            liste[where+1] = liste[where]
            where -= 1
        liste[where+1] = to_insert
        print(liste)
    return liste

liste = [5,1,7,2,6,4,3]
tri_par_insertion(liste)
```

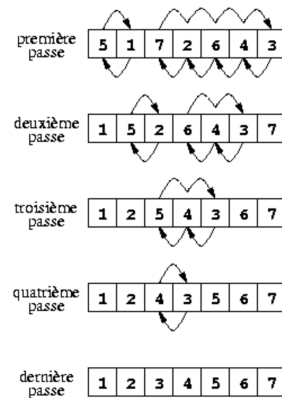
```
[1, 5, 7, 2, 6, 4, 3]
[1, 5, 7, 2, 6, 4, 3]
[1, 2, 5, 7, 6, 4, 3]
[1, 2, 5, 6, 7, 4, 3]
[1, 2, 4, 5, 6, 7, 3]
[1, 2, 3, 4, 5, 6, 7]
```

```
[2]: [1, 2, 3, 4, 5, 6, 7]
```

2.3 Tri à bulles

Le tri à bulles s'appuie sur des permutations répétées d'éléments contigus qui ne sont pas dans l'ordre afin de placer le plus grand élément à sa position finale. Ce processus est répété (en ignorant la partie déjà triée) jusqu'à ce que tout soit trié.

2.3.1 Illustration



2.3.2 Exemple de code

```
[3]: def tri_à_bulles(liste):
    for i1 in range(len(liste)-1):
        changed = False # flag pour optimiser, la liste est triée si aucune
        ↳ permutation n'est détectée.
        for i2 in range(len(liste)-1-i1):
            if liste[i2]>liste[i2+1]:
                liste[i2], liste[i2+1] = liste[i2+1], liste[i2]
                changed = True
        print(liste)
        if not changed:
            break

liste = [5,1,7,2,6,4,3]
tri_à_bulles(liste)
```

```
[1, 5, 2, 6, 4, 3, 7]
[1, 2, 5, 4, 3, 6, 7]
[1, 2, 4, 3, 5, 6, 7]
[1, 2, 3, 4, 5, 6, 7]
[1, 2, 3, 4, 5, 6, 7]
```

3 Tri par comptage

Le tri par comptage est un algorithme de tri non comparatif utilisé principalement lorsque les valeurs à trier sont dans un domaine de nombres limité. Il est très efficace lorsque l'écart entre la plus petite et la plus grande valeur est raisonnable par rapport à la taille des données.

3.1 Principe

- Trouver la valeur minimale et la valeur maximale.

- Créer une liste de comptage où chaque indice représente une valeur possible de la liste originale, et stocker le nombre d'occurrences de chaque valeur.
- Parcourir la liste de comptage pour reconstruire la liste triée.

3.2 Exemple de code

```
[4]: def tri_par_comptage(liste):
    mini, maxi = min(liste), max(liste)
    comptage = [0 for _ in range(maxi-mini+1)]
    for i in range(mini, maxi+1):
        comptage[i-mini] = liste.count(i)
    print('comptage =', comptage)
    resultat = []
    for x in comptage:
        resultat.extend(x*[mini])
        mini += 1
    return resultat

liste = [1,1,1,3,3,5,7,7,7,7]
tri_par_comptage(liste)
```

```
comptage = [3, 0, 2, 0, 1, 0, 4]
```

```
[4]: [1, 1, 1, 3, 3, 5, 7, 7, 7, 7]
```

4 Fonctions de tri prédéfinies en Python

- Un objet de type *list* a une méthode *sort()* qui trie en modifiant la liste elle-même.
- Pour n'importe quel itérable, on peut utiliser la fonction *sorted()*.

4.1 Exemples

```
[5]: nombres = [7,3,2,1,5,4,6]

nombres.sort()    # modifie la liste elle-même

print(nombres)
```

```
[1, 2, 3, 4, 5, 6, 7]
```

```
[6]: nombres = [7,3,2,1,5,4,6]

sorted(nombres)    # renvoie une autre liste triée sans modifier la liste
                  ↪originelle
```

```
[6]: [1, 2, 3, 4, 5, 6, 7]
```

```
[7]: texte = 'chose à trier!'

sorted(texte)    # renvoie une liste des caractères triés selon leurs codes
```

```
[7]: [' ', ' ', '!', 'c', 'e', 'e', 'h', 'i', 'o', 'r', 'r', 's', 't', 'à']
```

```
[8]: contacts = {'Sarah' : 153455, 'Ahmad' : 205522, 'Ibrahim' : 212100}

sorted(contacts)    # renvoie une liste des clés triées
```

```
[8]: ['Ahmad', 'Ibrahim', 'Sarah']
```

Les deux fonctions ont :

- un paramètre *reverse* qui vaut par défaut *False* pour un tri en ordre croissant et *True* pour un ordre décroissant.
- un paramètre *key* afin de spécifier une fonction qui prend un seul argument et qui peut être appelée sur chaque élément de la liste. Ainsi, les éléments de la liste sont triés en comparant les résultats renvoyés par cette fonction.

```
[9]: nombres = [7,3,2,1,5,4,6]

nombres.sort(reverse=True)

print(nombres)
```

```
[7, 6, 5, 4, 3, 2, 1]
```

```
[10]: def inverse(x):
        return 1/x

nombres = [7,3,2,1,5,4,6]

sorted(nombres, key=inverse)

#ou bien

sorted(nombres, key=lambda x:1/x)
```

```
[10]: [7, 6, 5, 4, 3, 2, 1]
```

```
[11]: transactions = [
        ['client_1', '17/03/2025', 1000],
        ['client_2', '17/03/2025', 2000],
        ['client_1', '10/03/2025', 1000],
        ['client_3', '02/03/2025', 1500]
    ]

sorted(transactions, key=lambda x:x[2])
```

```
[11]: [['client_1', '17/03/2025', 1000],  
      ['client_1', '10/03/2025', 1000],  
      ['client_3', '02/03/2025', 1500],  
      ['client_2', '17/03/2025', 2000]]
```

Dans ce cas on trie les transactions par montant. L'algorithme de tri est stable car l'ordre chronologique est préservé pour les montants égaux. Avec un tri non stable, on pourrait avoir 1000 (10/03) avant 1000 (17/03), ce qui change l'interprétation des données.